MODULE 2: TRANSPORT LAYER

UDP CHECKSUM

- **UDP checksum** is used for **error detection**.
- It checks whether the bits in a UDP segment were changed during transmission.

At the Sender

1. Divide the UDP segment into **16-bit words**.
2. Add all the 16-bit words using **1's complement addition**.
3. If there is an overflow, **wrap it around** and add it to the result.
4. Take the **1's complement** of the final sum.
5. The result is placed in the **checksum field**.

At the Receiver

- The receiver adds **all 16-bit words, including the checksum**.
- If there is **no error**, the result is:
  1111111111111111
- If the result is different, an **error has occurred**.

Example

Step 1: Add the 16-bit words

```
1001 0110 1010 1100
0011 0101 1100 0011
-------------------
1100 1100 0110 1111

1100 1100 0110 1111
1110 0001 0101 0110
-------------------
1 1011 1101 1100 0101
```

There is a **carry** outside 16 bits.

Step 2: Add the carry back

```
1011 1101 1100 0101
0000 0000 0000 0001
-------------------
1011 1101 1100 0110
```

Step 3: Take 1's complement

Change 0 → 1 and 1 → 0:

```
1011 1101 1100 0110
        ↓
0100 0010 0011 1001
```

Answer

UDP Checksum = `0100 0010 0011 1001`

Why does UDP need a checksum?

- transferring Some links may **not provide error checking**.
- Errors can also occur while a segment is **stored in a router's memory**.
- Therefore, UDP provides **end-to-end error detection**.

UDP can detect errors, but it does not recover from them.

- A damaged segment may be **discarded**, or
- It may be given to the application with a **warning**.

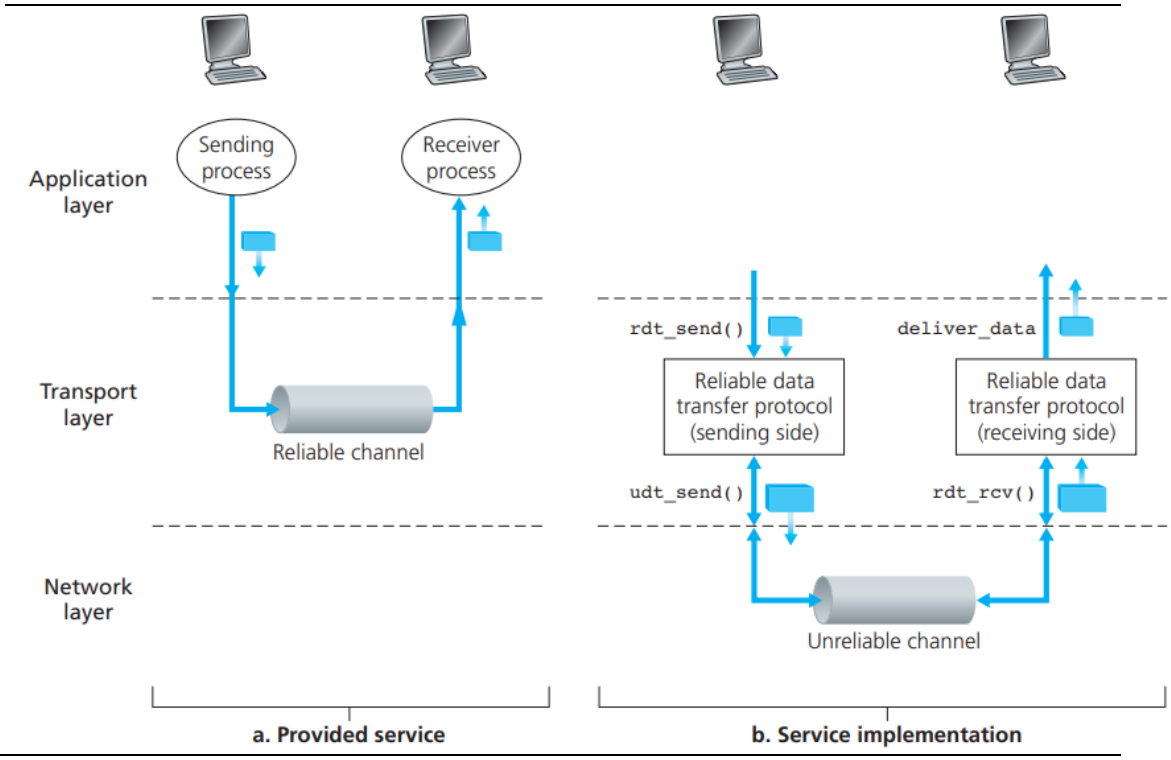
PRINCIPLES OF RELIABLE DATA TRANSFER

Reliable Data Transfer (RDT) means data **correctly, without loss or corruption, and in the same order** as it was sent.

- RDT is important in the **transport, link, and application layers**.
- The layer below RDT may be **unreliable**. For example, **TCP provides reliable data transfer over the unreliable IP layer**.

- RDT handles problems such as **bit corruption and packet loss**.
- The underlying channel is assumed **not to change the order of packets**, although some packets may be lost.

  rdt_send() – sends data from the **upper layer to the RDT sender**.
  udt_send() – sends packets through the **unreliable channel**.
  rdt_rcv() – receives packets from the **unreliable channel**.
  deliver_data() – delivers the received data to the **upper layer**.
- Although data transfer is **unidirectional**, control packets such as **ACKs can travel in both directions** between sender and receiver.



BUILDING A RELIABLE DATA TRANSFER

Reliable Data Transfer over a Perfectly Reliable Channel: rdt1.0

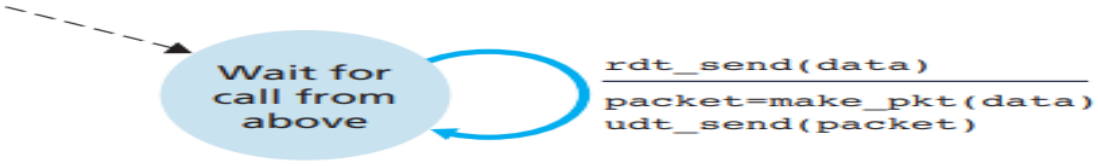
• rdt1.0 is the simplest reliable data transfer protocol because the channel is completely reliable.
• There is no packet loss, corruption, or reordering.
• The sender and receiver each have only one state.

Sender:
• rdt_send(data) – receives data from the upper layer.
• make_pkt(data) – creates a packet containing the data.
• udt_send(packet) – sends the packet to the receiver.

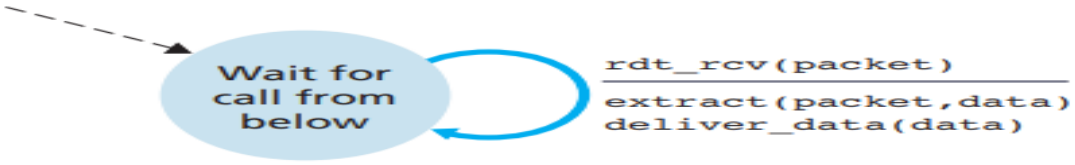
Receiver:
• rdt_rcv(packet) – receives the packet.
• extract(packet, data) – extracts data from the packet.
• deliver_data(data) – delivers the data to the upper layer.

• No ACKs or control packets are needed because the channel is perfectly reliable.
• Data flows only from the sender to the receiver.
• No flow control is needed because the receiver can receive data as fast as the sender sends it.

Simple flow:
Data → Make Packet → Send → Receive → Extract Data → Deliver



a. rdt1.0: sending side



b. rdt1.0: receiving side

Reliable Data Transfer over a Channel with Bit Errors
• Bits in a packet may be corrupted during transmission.
• Packets are assumed to be not lost or reordered.
• They arrive in the same order as sent.
• ARQ provides reliable transfer using:
• Error detection
• ACK/NAK feedback
• Retransmission

rdt2.0
• rdt2.0 handles corrupted packets.
• The sender adds a checksum to the packet.
• The receiver checks the packet for errors.
• ACK means the packet is correct.
• NAK means the packet is corrupted.
• ACK → sender sends the next packet.
• NAK → sender sends the same packet again.
• rdt2.0 is a stop-and-wait protocol.

Sender:
rdt_send(data) → make_pkt(data, checksum) → udt_send(sndpkt) → wait
ACK → send next data
NAK → send same packet again

Receiver:
rdt_rcv(rcvpkt) → check packet
Correct → extract(data) → deliver_data(data) → make_pkt(ACK) → udt_send(ACK)
Corrupted → make_pkt(NAK) → udt_send(NAK)

Problem:
• ACK/NAK can also be corrupted.
• The sender cannot know the receiver's actual result.
• Retransmission may create duplicate packets.

• Sequence numbers solve this problem.



Figure 3.10 ♦ rdt2.0–A protocol for a channel with bit errors

rdt2.1

• rdt2.1 adds sequence numbers.
• Sequence numbers are 0 and 1.
• They help identify duplicate packets.
• A duplicate packet is not delivered again.
• Corrupted feedback causes retransmission.

Sender:
rdt_send(data) → make_pkt(0/1, data, checksum) → udt_send(sndpkt) → wait ACK → send next packet
NAK / corrupted ACK or NAK → send same packet again

Receiver:
rdt_rcv(rcvpkt) → check error + sequence number
New & correct → extract(data) → deliver_data(data) → send ACK
Duplicate → do not deliver → send ACK
Corrupted → send NAK

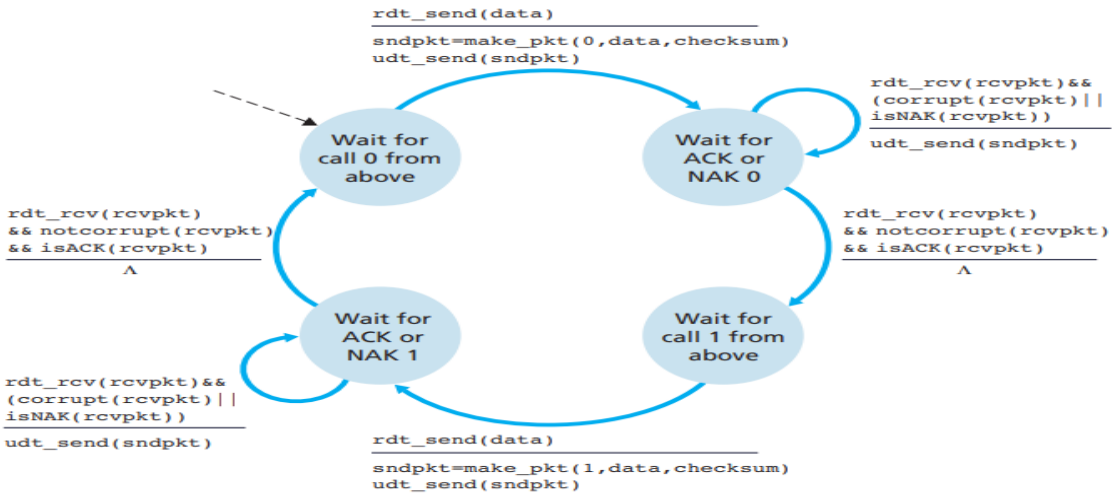


Figure 3.11 ♦ rdt2.1 sender

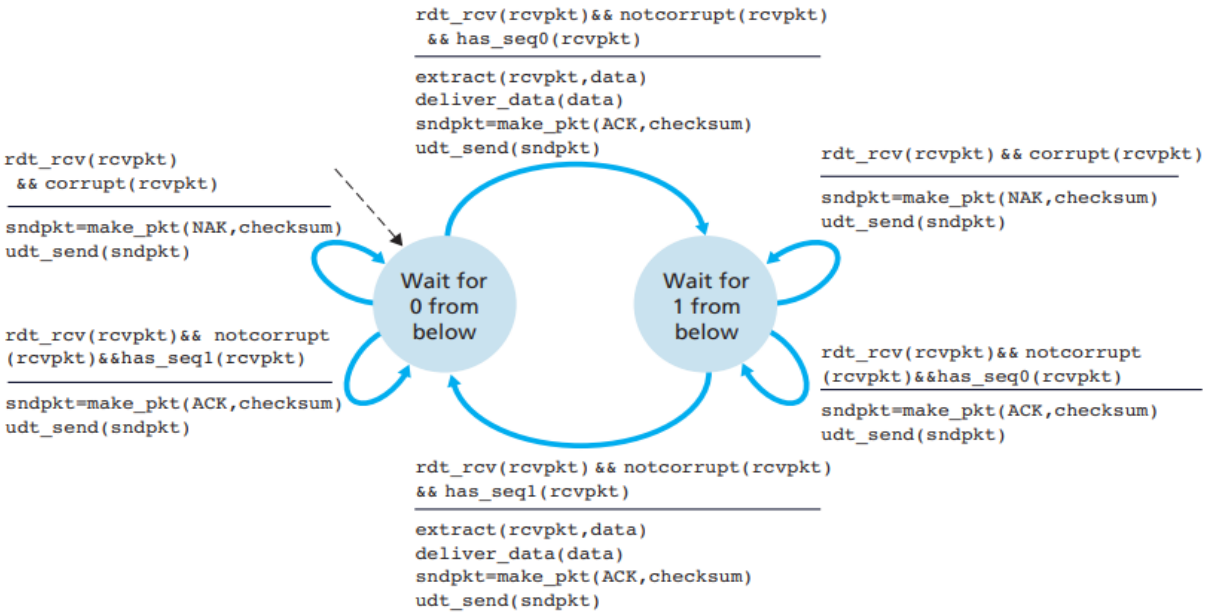


Figure 3.12 ♦ rdt2.1 receiver

rdt2.2

- rdt2.2 is similar to rdt2.1.
- It does not use NAK.
- It uses ACK with a sequence number.
- ACK can be ACK 0 or ACK 1.
- Duplicate ACK indicates a problem with the current packet.
- The sender retransmits the current packet.

Sender:

rdt_send(data) → make_pkt(0/1, data, checksum) → udt_send(sndpkt) → wait
Correct ACK → send next packet
Duplicate ACK → send same packet again

Receiver:

rdt_rcv(rcvpkt) → check error + sequence number
Correct → extract(data) → deliver_data(data) → make_pkt(ACK, 0/1) → udt_send(ACK)
Corrupted / wrong → send ACK for last correct packet

Simple Difference

rdt2.0 → ACK + NAK
rdt2.1 → ACK + NAK + Sequence Number
rdt2.2 → ACK + Sequence Number, No NAK

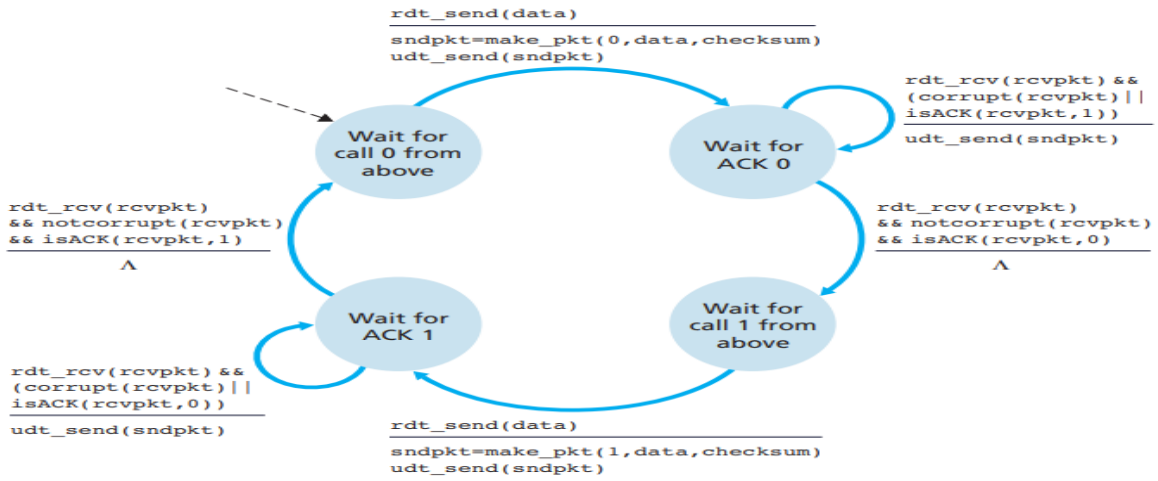


Figure 3.13 ♦ rdt2.2 sender

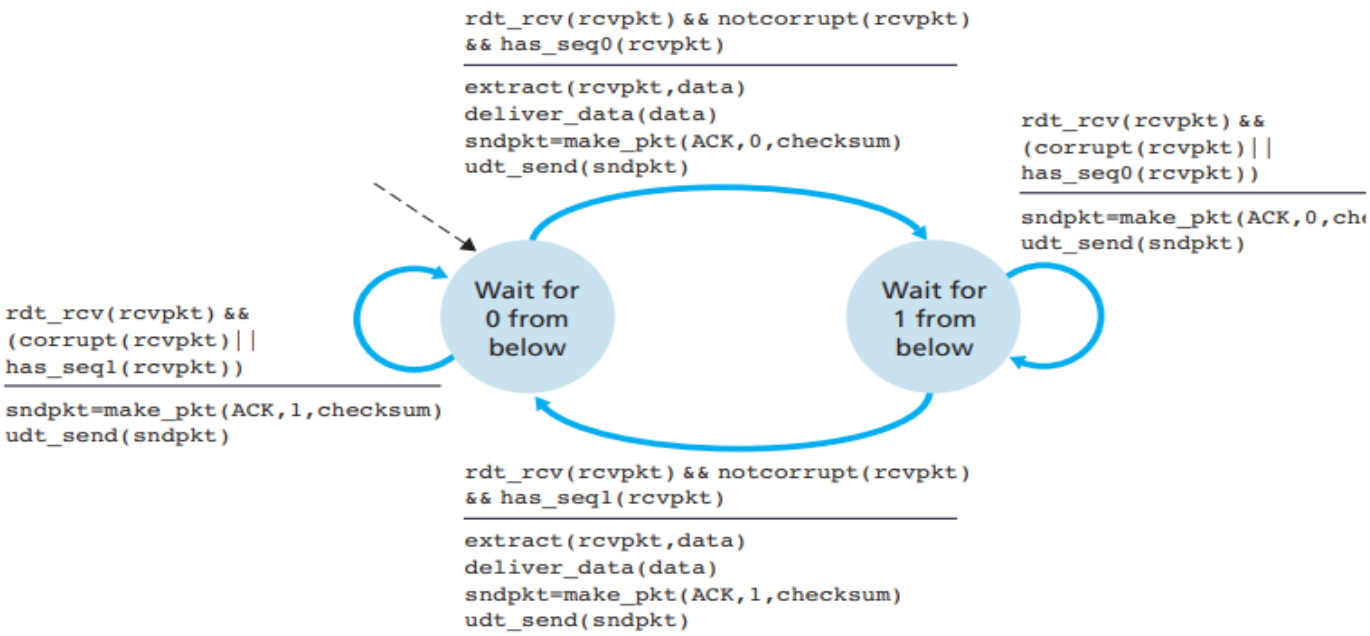


Figure 3.14 ♦ rdt2.2 receiver

rdt3.0

• rdt3.0 works when packets can be corrupted or lost.
• It uses sequence numbers, ACKs, checksums, and a timer.
• Sequence numbers are 0 and 1.
• The sender starts a timer after sending a packet.
• If the ACK does not arrive before the timer expires, the sender retransmits the packet.
• If a packet is lost, the timer helps the sender detect the problem.
• Because sequence numbers alternate between 0 and 1, rdt3.0 is also called the Alternating-Bit Protocol.

Sender:
rdt_send(data) → make_pkt(seq, data, checksum) → udt_send(sndpkt) → start timer
Correct ACK → stop timer → change seq (0↔1) → send next data
Corrupted/wrong ACK → ignore → wait
Timeout → retransmit same packet → restart timer

Receiver:
rdt_rcv(rcvpkt) → check packet + sequence number
Correct & expected → extract(data) → deliver_data(data) → send ACK → wait for next seq
Corrupted / duplicate → do not deliver → send ACK for last correct packet

Working:
Sender → Data packet → Receiver
Receiver → ACK → Sender
No ACK → Timer expires → Sender retransmits
Duplicate packet → Receiver detects using sequence number → does not deliver again

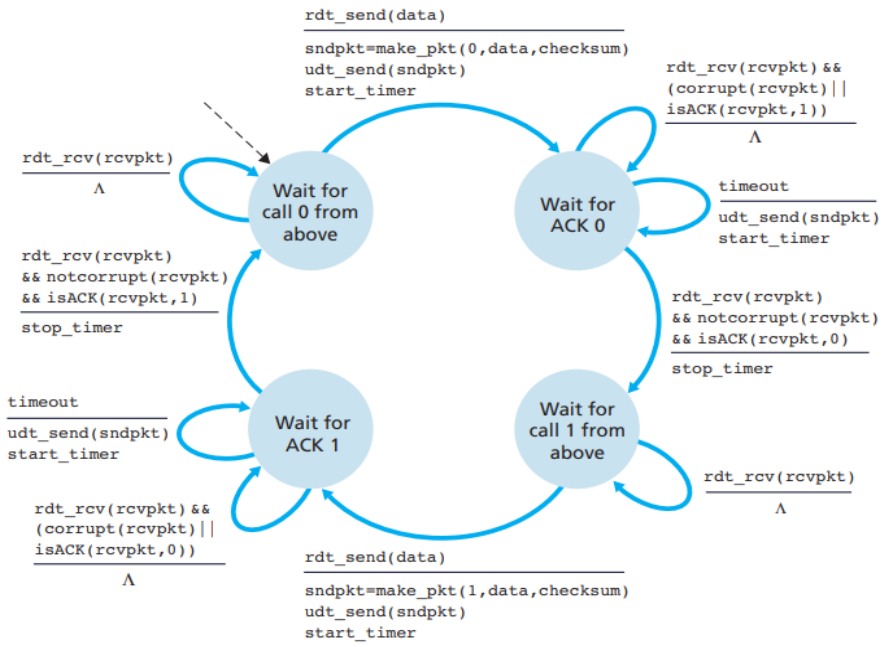


Figure 3.15 ♦ rdt3.0 sender

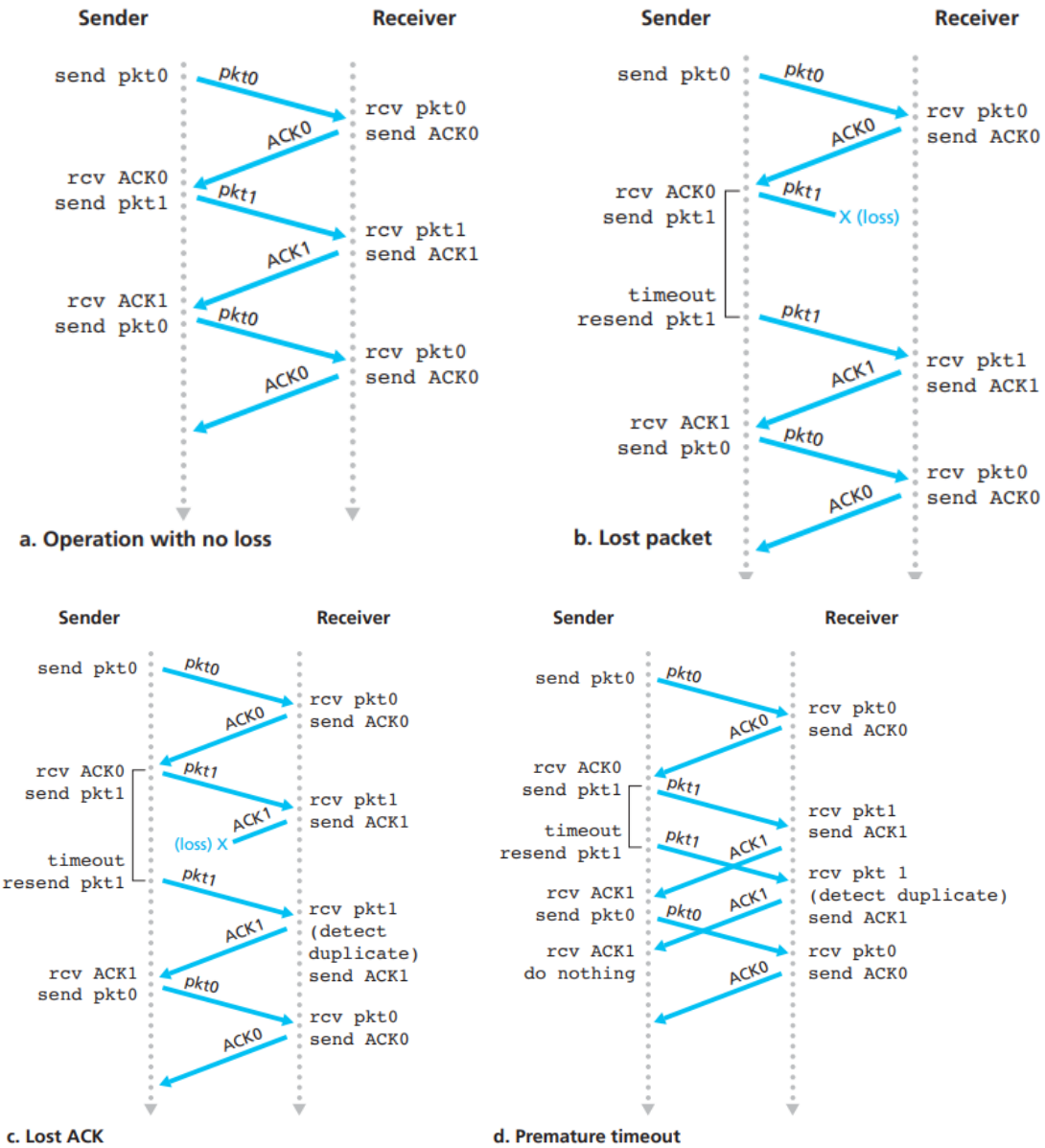


Figure 3.16 ♦ Operation of rdt3.0, the alternating-bit protocol

Pipelined Reliable Data Transfer

• rdt3.0 uses a stop-and-wait approach, so the sender sends one packet and waits for its ACK before sending the next packet.

• This can cause poor performance because the sender remains idle while waiting for the ACK.

Example:

R = Transmission rate = 1 Gbps

L = Packet size = 1000 bytes = 8000 bits

RTT = Round-Trip Time = 30 ms

Transmission time:

dtrans = L / R

= 8000 / 10^9

= 8 μs = 0.008 ms

Sender utilization:

Usender = (L/R) / (RTT + L/R)

≈ 0.027%

• Thus, although the link has a rate of 1 Gbps, the sender uses it for only a very small amount of time.

Pipelining

• Pipelining allows the sender to send multiple packets without waiting for an ACK for each packet.

Stop-and-wait:

Packet 0 → Wait → Packet 1 → Wait → Packet 2

Pipelining:

Packet 0 → Packet 1 → Packet 2 → Packet 3

• While Packet 0 is travelling to the receiver, the sender can send Packet 1 and Packet 2.

• Therefore, several packets can be in transit at the same time.

• ACKs can arrive while the sender continues sending new packets.

• This keeps the link busy and improves utilization and throughput.

Pipelining requires:

• Larger sequence-number range.

• Sender buffer for unacknowledged packets.

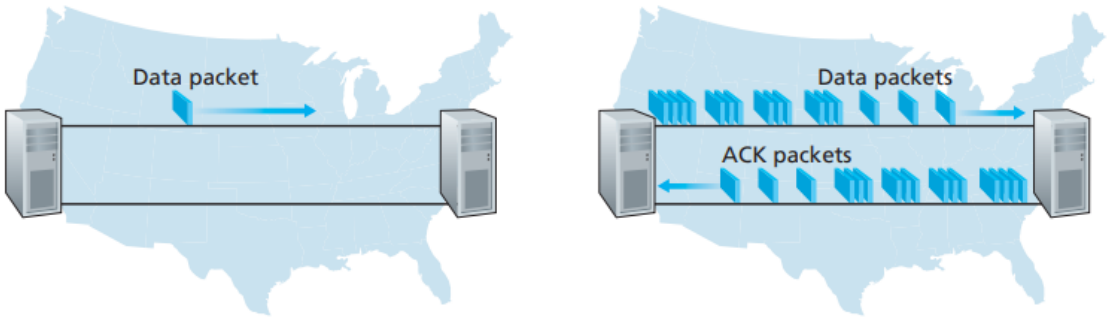
• Receiver may also need to buffer packets.

• Proper handling of lost, corrupted, or delayed packets.

Two important pipelined protocols are:

• **Go-Back-N (GBN)** – retransmits the lost packet and the packets after it.

• **Selective Repeat (SR)** – retransmits only the lost or corrupted packet.



a. A stop-and-wait protocol in operation

b. A pipelined protocol in operation

Figure 3.17 ♦ Stop-and-wait versus pipelined protocol

GO-BACK-N (GBN) PROTOCOL

1. What is Go-Back-N?

Go-Back-N is a **pipelined reliable data transfer protocol**.

- The sender can send **multiple packets without waiting for ACKs**.
- But only **N unacknowledged packets** can be in the pipeline.
- **N = Window Size**.
- The window **slides forward** as ACKs are received.

2. Sender Window

The sender uses two important variables:

- **base** → sequence number of the **oldest unacknowledged packet**.
- **nextseqnum** → sequence number of the **next packet to be sent**.

The sequence numbers are divided into four parts:

Range	Meaning
0 to base-1	Packets already ACKed
base to nextseqnum-1	Sent but not ACKed
nextseqnum to base+N-1	Can be sent immediately
base+N and above	Cannot be sent yet

Example: If base = 2, nextseqnum = 5, and N = 4:

- 0,1 → already ACKed
- 2,3,4 → sent, waiting for ACK
- 5,6 → can be sent
- 7 and above → cannot be sent yet

3. Sliding Window

When an ACK arrives, the window **moves forward**.
For example:
[0 1 2 3] → ACK 0 → [1 2 3 4]
So, GBN is also called a **Sliding Window Protocol**.

4. Three Main Events at Sender

A. Data arrives from upper layer

If the window is **not full**:

1. Create packet.
2. Add sequence number.
3. Send packet.
4. Increase nextseqnum.

If the window is full, the sender **cannot send a new packet**.

B. ACK is received

GBN uses **cumulative ACKs**.
For example:
ACK 3 means packets **0, 1, 2, and 3** have been received correctly.
The sender then moves base forward.

C. Timeout occurs

If a packet is lost or its ACK is lost:

- Timer expires.

- Sender retransmits **all unacknowledged packets** starting from base.

This is why it is called **Go-Back-N**.

5. Receiver Operation

The receiver expects packets **in order**.

If the expected packet arrives correctly:

1. Accept the packet.
2. Deliver its data to the upper layer.
3. Send an ACK.
4. Move to the next expected sequence number.

If an **out-of-order packet** arrives:

- Discard it.
- Send an ACK for the **last correctly received in-order packet**.

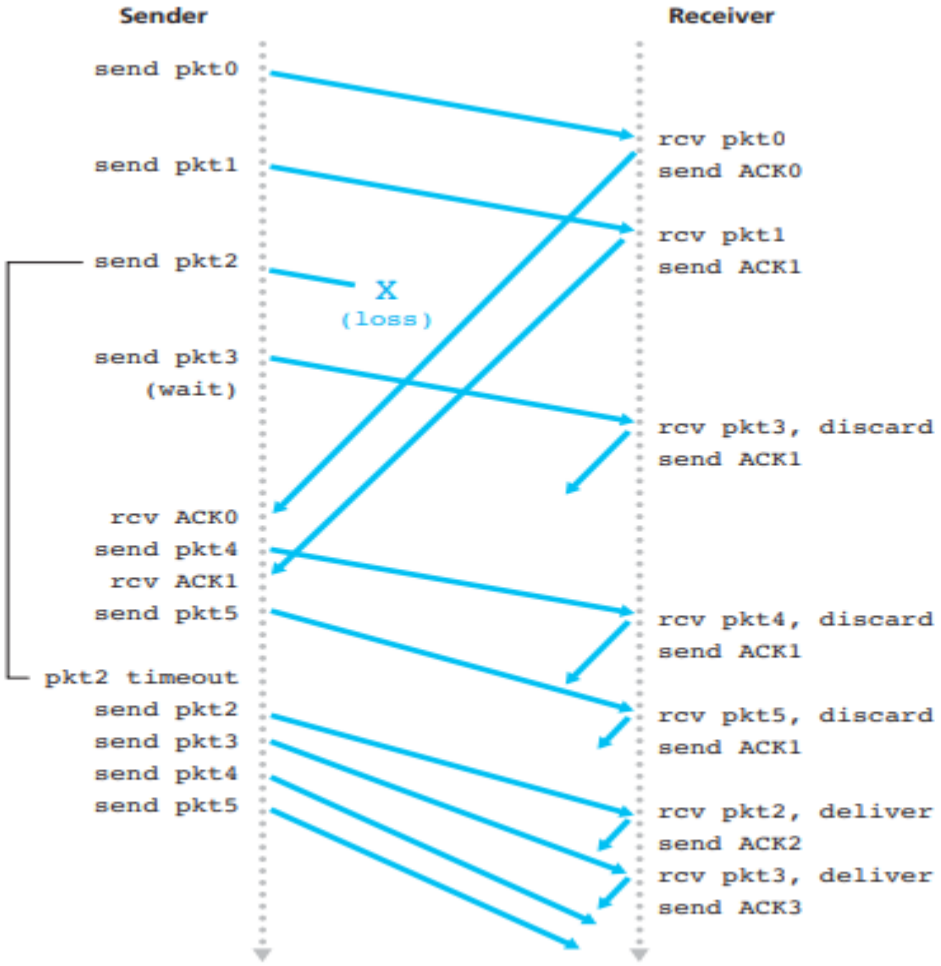


Figure 3.22 ♦ Go-Back-N in operation

Selective Repeat (SR)

- **GBN problem:** If one packet is lost, GBN retransmits that packet **and all following packets**, even if they were received correctly.
- This can cause **many unnecessary retransmissions**, especially when the window size and bandwidth-delay product are large.
- **SR retransmits only the packets that are lost or corrupted.**

- The sender uses a **window of N packets** to limit the number of unacknowledged packets.
- The receiver **individually acknowledges every correctly received packet**, even if it arrives out of order.
- **Out-of-order packets are buffered** at the receiver.
- When the missing packet arrives, the buffered packets are delivered **in the correct order** to the upper layer.

Example:
If packet **2 is lost**, but packets **3, 4, and 5** arrive:

- Receiver stores 3, 4, and 5 in its buffer.
- Sender retransmits only packet **2**.
- When packet 2 arrives, packets **2, 3, 4, and 5** are delivered in order.

Main idea:
GBN → retransmit from the lost packet onward.
SR → retransmit only the lost/corrupted packet.

SR Re-ACK

- send_base is the **oldest unacknowledged packet** at the sender.
- Suppose the receiver has already received send_base, but its **ACK is lost**.
- The sender does not know this, so it **retransmits send_base**.
- The receiver receives it again and **sends the ACK again**.
- This ACK tells the sender that send_base was already received, so the **sender moves its window forward**.
- Therefore, the **sender's and receiver's windows may be different**.

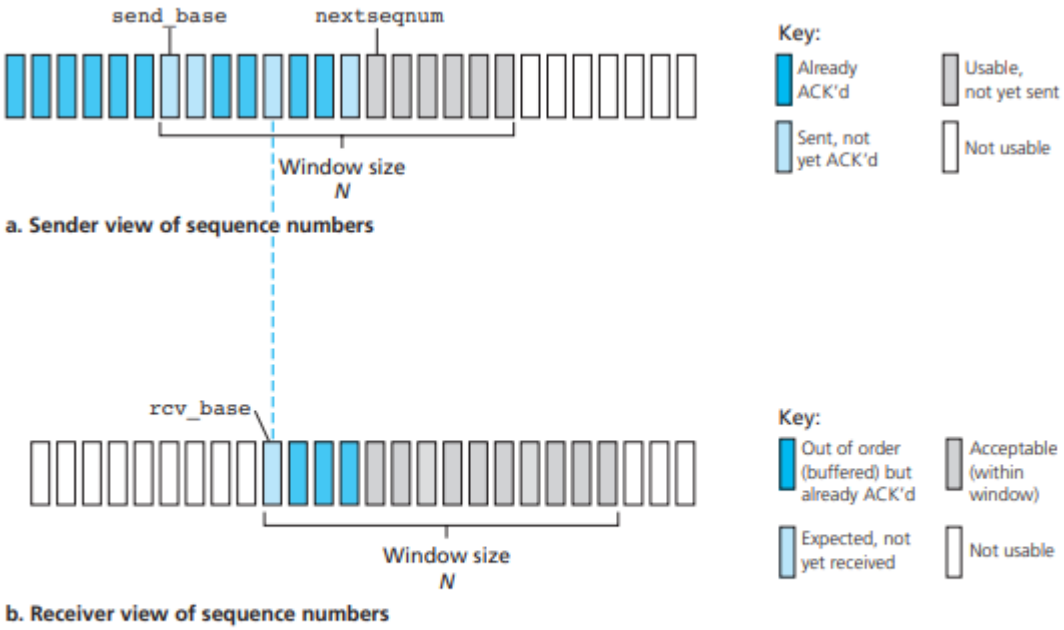


Figure 3.23 ♦ Selective-repeat (SR) sender and receiver views of sequence-number space

SR Sender

1. Data received from above

- Check the packet's **sequence number**.

- If it is within the sender's window → **packetize and send**.
- Otherwise → **buffer it** for later.

2. Timeout

- Each packet has its **own timer**.
- When a packet times out, **only that packet is retransmitted**.

3. ACK received

- Mark that packet as **ACKed**.
- If it is send_base, move the **window forward**.
- Send any new packets that now fall inside the window.

SR Receiver

1. Packet within receiver window

- Send an **ACK**.
- If new, **buffer the packet**.
- If it is rcv_base, deliver it and any **consecutive buffered packets** to the upper layer.
- Move the receiver window forward.

Example: If rcv_base = 2 and packets **2, 3, 4, 5** are available → deliver **2, 3, 4, 5**.

2. Packet before receiver window

- If it was already received earlier → **send the ACK again**.

3. Otherwise

- **Ignore the packet.**

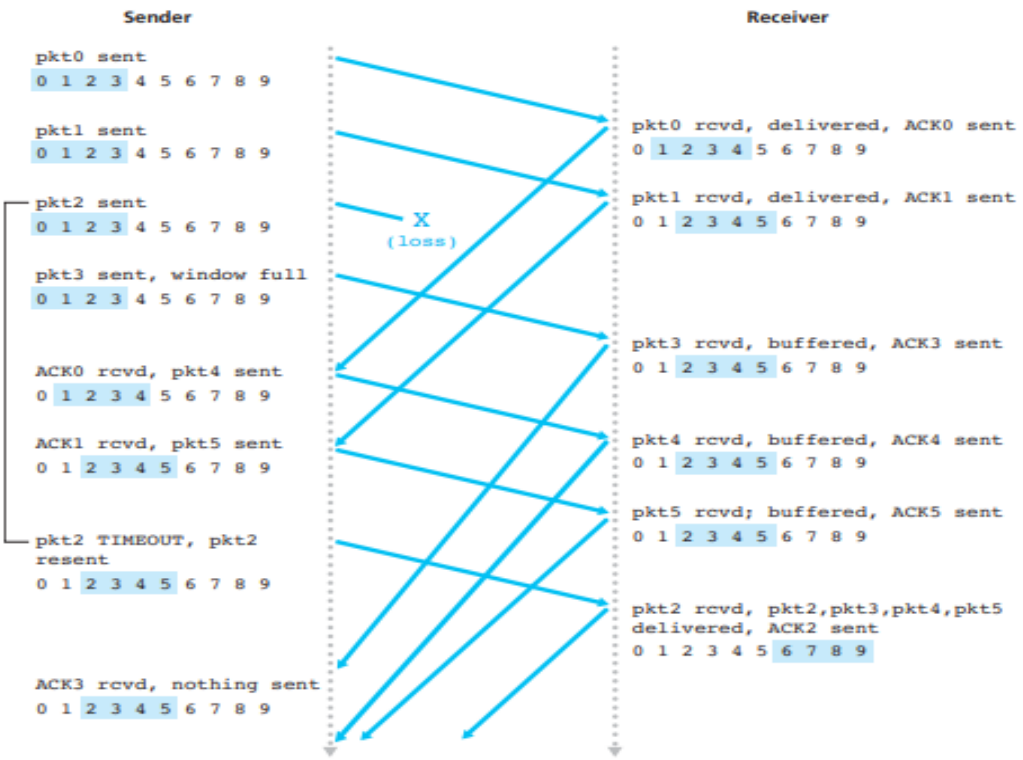


Figure 3.26 ♦ SR operation

SR window size must be limited.

- Sequence numbers are reused.
- If the window is too large, the receiver can **confuse an old retransmitted packet with a new packet**.
- Therefore:

SR Window Size ≤ ½ × Sequence Number Space

Example:
Sequence numbers = **0, 1, 2, 3** → 4 numbers
Maximum window size = **2**

TCP CONNECTION

TCP Connection

- **TCP (Transmission Control Protocol)** is **connection-oriented** because the client and server must first **handshake** before data transfer.
- They exchange preliminary segments and **initialize TCP state variables**.
- TCP connection state is maintained only at the **two end systems**, not in routers or switches.
- TCP is **not a physical or virtual circuit**; routers see **datagrams**.

Connection Establishment

- The **client** requests a connection using:
  clientSocket.connect((serverName, serverPort))
- TCP performs a **three-way handshake**:
  1. Client → Server: sends a special TCP segment.
  2. Server → Client: sends a response segment.
  3. Client → Server: sends a confirmation segment.
- The first two segments carry **no application data**; the third may carry data.

Data Transfer and Socket

- The application sends data through the **socket**, which acts as a **door between the application and TCP**.
- TCP stores data in the **send buffer**, forms TCP segments, and passes them to **IP (Internet Protocol)**.
- At the receiver, data is placed in the **receive buffer**, and the application reads it through its **socket**.
- Each side has its own **socket, send buffer, receive buffer, and TCP state variables**.

MTU (Maximum Transmission Unit)

- MTU = **maximum size of a link-layer frame** that can be transmitted over a link.
- Think of it as the **maximum size of the packet/container that the link can carry**.

MSS (Maximum Segment Size)

- MSS = **maximum amount of application data carried inside one TCP segment**.
- MSS is smaller than MTU because the TCP segment also has **TCP and IP headers**.

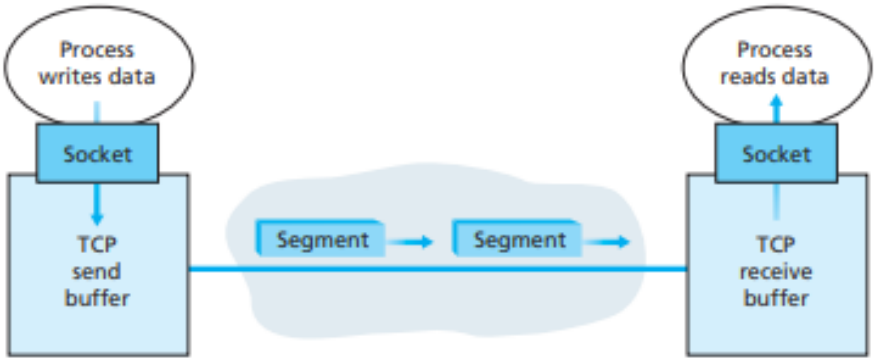
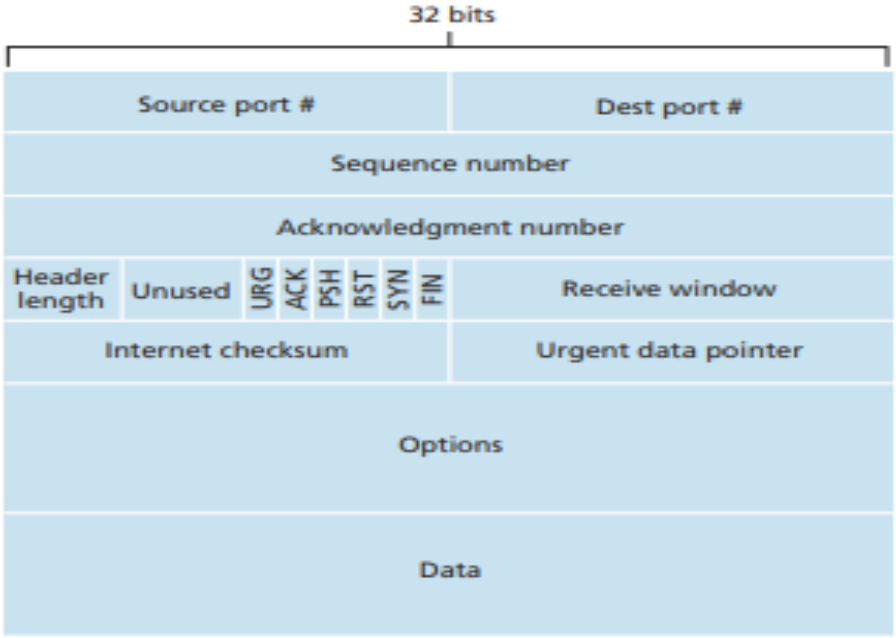


Figure 3.28 ♦ TCP send and receive buffers

TCP SEGMENT STRUCTURE

A **TCP segment** has two main parts:

1. **Header** – contains control and connection information.
2. **Data field** – contains application data.



Important TCP Header Fields

- **Source Port & Destination Port:** Identify the sending and receiving applications.
- **Sequence Number (32 bits):** Identifies the position of data bytes; helps **reliable data transfer**.
- **Acknowledgment Number (32 bits):** Indicates the next byte the receiver expects; used for acknowledgments.
- **Receive Window (16 bits):** Used for **flow control**; tells how many bytes the receiver can accept.
- **Checksum:** Used to detect errors in the segment.
- **Header Length:** Indicates the length of the TCP header. Normally, the header is **20 bytes**.
- **Options:** Optional fields used for things such as **MSS negotiation, window scaling, and timestamps**.
- **Flags:** Control connection and data transfer:
    - **ACK:** Acknowledgment is valid.
    - **SYN:** Used to establish a connection.
    - **FIN:** Used to terminate a connection.
    - **RST:** Resets the connection.
    - **PSH:** Requests immediate delivery of data to the application.
    - **URG:** Indicates urgent data.

Data Field and MSS

- **MSS (Maximum Segment Size)** limits the maximum amount of application data in one segment.
- Large data, such as a file, is divided into **MSS-sized chunks**.
- Small interactive applications may send much smaller chunks.

TCP Sequence Numbers and Acknowledgment Numbers

1. Sequence Number

- TCP treats data as an **ordered stream of bytes**, not as a series of segments.
- The **sequence number** of a segment is the number of the **first byte of data** in that segment.

Example:

If MSS = 1,000 bytes and the first byte is numbered 0:

- Segment 1 → Sequence number **0**
- Segment 2 → Sequence number **1,000**
- Segment 3 → Sequence number **2,000**

2. Acknowledgment Number

- The **acknowledgment number** indicates the **next byte the receiver expects**.
- Example: If Host A has received bytes **0–535**, it expects byte **536**, so:
  ACK = 536

3. Cumulative Acknowledgment

- TCP uses **cumulative acknowledgments**.
- If bytes 0–535 and 900–1000 are received, but **536–899 are missing**, the ACK is still **536**.
- TCP acknowledges only up to the **first missing byte**.

4. Out-of-Order Segments

- A segment may arrive before an earlier segment.
- In practice, TCP usually **stores the out-of-order data** and waits for the missing bytes.
- This is more efficient than discarding it.

5. Initial Sequence Number

- Both sides of a TCP connection **randomly choose an initial sequence number**.
- This helps prevent old segments from a previous connection from being mistaken for segments of a new connection.

Remember:

Sequence Number = first byte in the segment
Acknowledgment Number = next byte expected

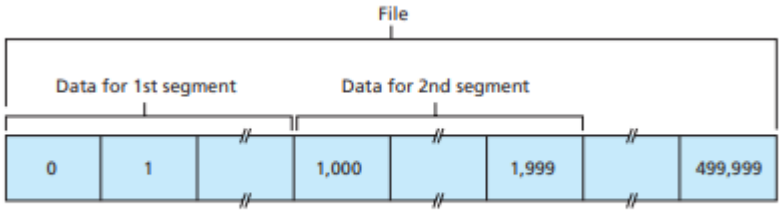


Figure 3.30 ♦ Dividing file data into TCP segments

Both sides of a TCP connection **randomly choose an initial sequence number**. This helps prevent an **old segment from a previous connection** from being mistaken for a segment of a **new connection**, even if the same port numbers are used.

TCP Reliable Data Transfer

1. Why Reliable Transfer is Needed

- **IP provides an unreliable best-effort service.** It does not guarantee:
  - Delivery of datagrams
  - In-order delivery
  - Data integrity
- Datagrams may be **lost, arrive out of order, or become corrupted**.
- Since TCP segments are carried inside IP datagrams, TCP must handle these problems.

2. TCP Reliable Data Transfer

- TCP provides **reliable data transfer on top of IP's unreliable service**.
- TCP ensures that the application receives a byte stream that is:
  - **Correct and uncorrupted**
  - **Without gaps**
  - **Without duplication**
  - **In the correct order**
- Thus, the received byte stream is the same as the one sent.

3. TCP Timer

- TCP uses **one retransmission timer**, even when several segments are unacknowledged.
- The timer is associated with the **oldest unacknowledged segment**.
- TimeoutInterval is calculated using **EstimatedRTT** and **DevRTT**.

4. TCP Sender

TCP sender mainly handles **three events**:

1. Data received from application

- TCP creates a segment with a **sequence number**.
- Sends it to IP.
- If the timer is not running, TCP **starts the timer**.

2. Timer timeout

- TCP **retransmits the oldest unacknowledged segment**.
- Then **restarts the timer**.

3. ACK received

- TCP checks the ACK value y against **SendBase**.
- **SendBase** = sequence number of the **oldest unacknowledged byte**.
- TCP uses **cumulative ACKs**, so ACK y means **all bytes before y have been received correctly and in order**.
- If y > SendBase, TCP updates **SendBase = y** and restarts the timer if unacknowledged segments remain.

Easy Flow

Data → Send segment → Start timer → ACK received → Move SendBase

If timeout → Retransmit oldest unacknowledged segment → Restart timer

Scenario 1: Lost Acknowledgment

- Host A sends a segment with **Seq = 92** containing **8 bytes** → bytes **92–99**.
- Host B receives it correctly and sends **ACK = 100**.
- The ACK is **lost**.
- Host A does not receive the ACK, so its **timer expires** and it retransmits the same segment.
- Host B recognizes that these bytes were **already received**, so it **discards the duplicate data** and sends **ACK = 100** again.

Main idea: Lost ACK → timeout → retransmission → receiver detects duplicate and discards it.

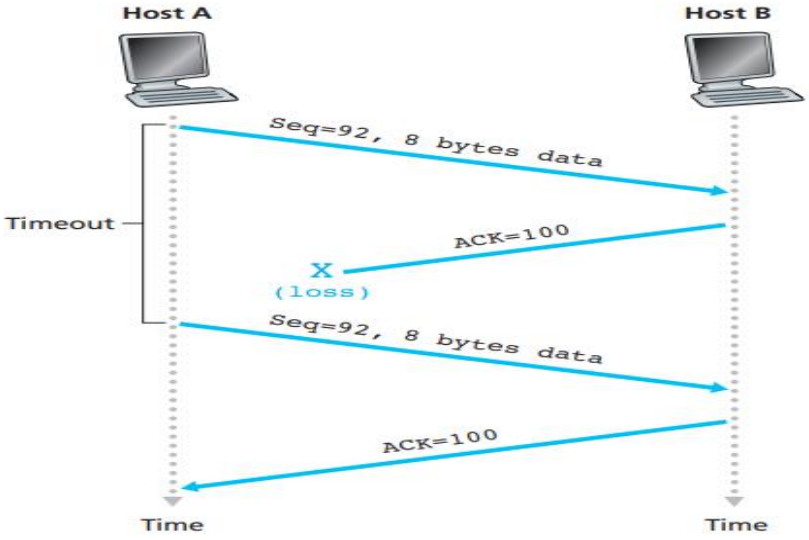


Figure 3.34 ◆ Retransmission due to a lost acknowledgment

Scenario 2: Lost ACKs for Two Segments

- Host A sends two segments:
  - **Seq = 92**, 8 bytes → ACK = **100**
  - **Seq = 100**, 20 bytes → ACK = **120**
- Both segments reach Host B correctly, but **both ACKs are lost**.
- Host A's timer expires, so it **retransmits the first segment (Seq = 92)** and restarts the timer.
- If **ACK = 120** arrives before the new timeout, it cumulatively acknowledges **both segments**.
- Therefore, the second segment **does not need to be retransmitted**.

Main idea: A single cumulative ACK can acknowledge multiple previously sent segments.

Scenario 3: ACK for Second Segment Arrives

- Host A sends two segments, as in Scenario 2.
- **ACK = 100 is lost**, but **ACK = 120 arrives before timeout**.
- ACK = 120 means Host B has received **all bytes up to 119**.
- Therefore, Host A **does not retransmit either segment**.

Main idea: A cumulative **ACK = 120 confirms both segments were received**.

Scenario 2

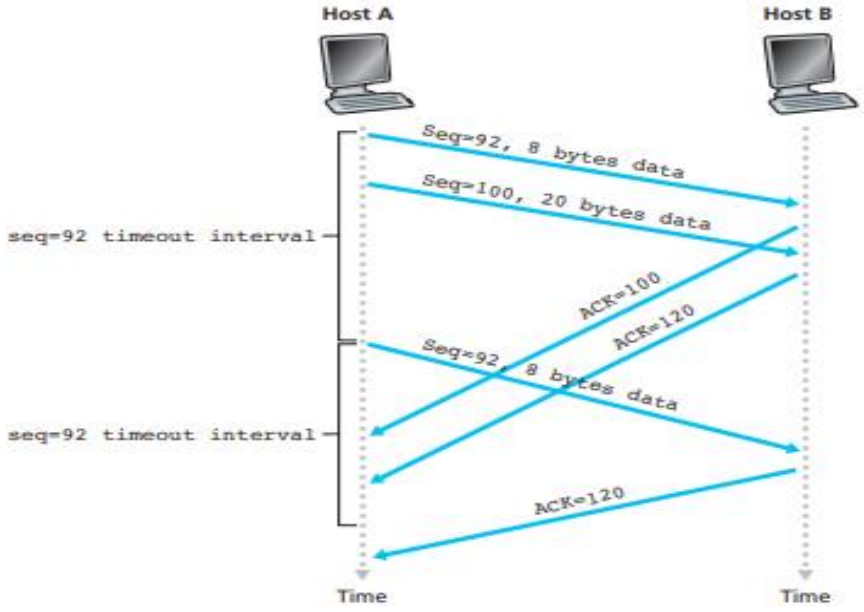


Figure 3.35 ♦ Segment 100 not retransmitted

Scenario 3

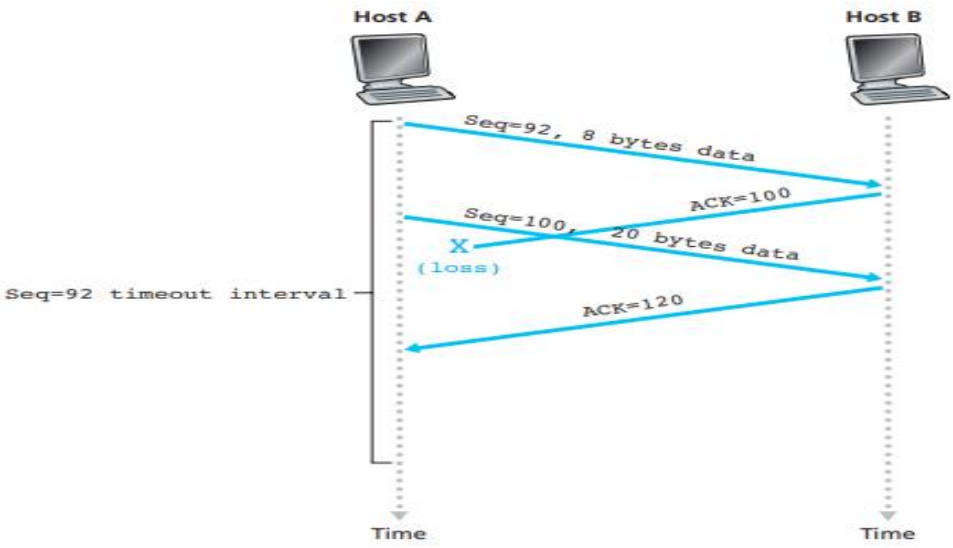


Figure 3.36 ♦ A cumulative acknowledgment avoids retransmission of the first segment

Doubling the Timeout Interval

- When a **timeout occurs**, TCP retransmits the **oldest unacknowledged segment**.
- After retransmission, TCP **doubles the timeout interval**:
  - 0.75 sec → 1.5 sec → 3 sec → 6 sec → …
- This is called **exponential backoff**.
- When a new timer starts because of **new data or a new ACK**, the timeout is recalculated using **EstimatedRTT and DevRTT**.
- **Purpose:** If timeout is caused by network congestion, longer intervals reduce repeated retransmissions and help prevent congestion from becoming worse.

Fast Retransmit

- Waiting for a timeout can take a long time.
- TCP can detect a lost segment earlier using **duplicate ACKs**.
- When a receiver gets an out-of-order segment, it sends a **duplicate ACK** for the last correctly received in-order byte.
- If the sender receives **3 duplicate ACKs**, it assumes the next segment is lost.
- TCP then **retransmits the missing segment immediately**, without waiting for timeout.

Example

If segment 2 is lost:

Segment 1 → Segment 3 → Segment 4 → Segment 5

Receiver repeatedly sends the ACK for Segment 1.

After **3 duplicate ACKs → sender fast retransmits Segment 2**.

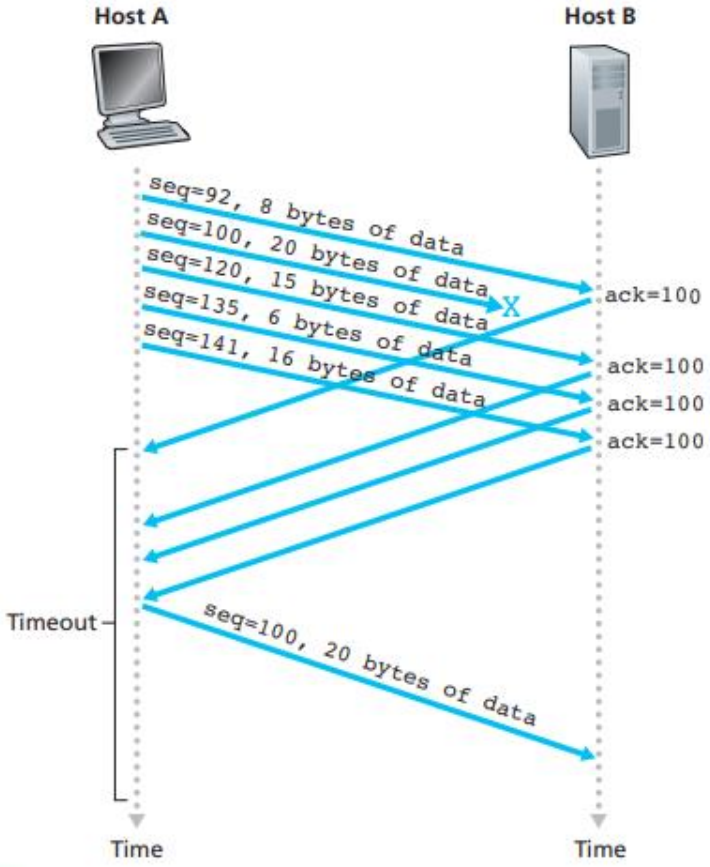


Figure 3.37 ♦ Fast retransmit: retransmitting the missing segment before the segment's timer expires

TCP FLOW CONTROL

Flow control is a service provided by TCP to prevent the sender from sending data faster than the receiver can process it. It prevents the **receiver's buffer from overflowing**.

Receive Buffer

When data arrives at the receiver, TCP stores the data temporarily in a **receive buffer**. The application reads data from this buffer when it is ready.

If the application is slow in reading the data, the buffer can become full. Therefore, TCP uses **flow control** to match the sender's sending rate with the receiver's reading rate.

Consider **Host A** sending data to **Host B**.

- **RcvBuffer** – Total size of the receive buffer at Host B.
- **LastByteRead** – Number of the last byte read from the buffer by the application.
- **LastByteRcvd** – Number of the last byte received from the network and stored in the buffer.

The amount of data currently occupying the buffer is:
LastByteRcvd − LastByteRead

Therefore:
LastByteRcvd − LastByteRead ≤ RcvBuffer

Receive Window (rwnd)
TCP uses a variable called the **receive window (rwnd)** to indicate how much free space is available in the receiver's buffer.

rwnd = RcvBuffer − (LastByteRcvd − LastByteRead)

The value of **rwnd changes with time** because the application continuously reads data from the buffer.

The receiver, Host B, informs Host A about its available buffer space by putting the current **rwnd value in the receive window field** of TCP segments.

Sender's Operation
Host A keeps track of:

- **LastByteSent** – Last byte sent by Host A.
- **LastByteAcked** – Last byte acknowledged by Host B.

The amount of unacknowledged data is:
LastByteSent − LastByteAcked

To avoid overflowing Host B's buffer, Host A ensures:
LastByteSent − LastByteAcked ≤ rwnd

Thus, the sender can send only as much unacknowledged data as the receiver has available buffer space.

When rwnd = 0
If Host B's buffer becomes full:
rwnd = 0

Host A then stops sending normal data because there is no free space in Host B's buffer.

But there is a small problem: after Host B's application reads some data, Host A needs to know that space has become available.

To solve this, TCP requires Host A to periodically send a **segment containing one byte of data** when rwnd = 0. Host B acknowledges this segment. When buffer space becomes available, the acknowledgment contains a **non-zero rwnd**, allowing Host A to continue sending.

Flow Control vs Congestion Control

Both can reduce the sender's transmission rate, but they have different purposes:

Flow Control	Congestion Control
Prevents receiver buffer overflow	Prevents network congestion
Based on receiver's available buffer space	Based on congestion in the network
Uses **rwnd**	Uses congestion-control mechanisms

UDP and Flow Control

UDP does not provide flow control. If UDP data arrives faster than the receiving application can read it, the buffer may become full. When the buffer overflows, incoming UDP segments may be **dropped**.

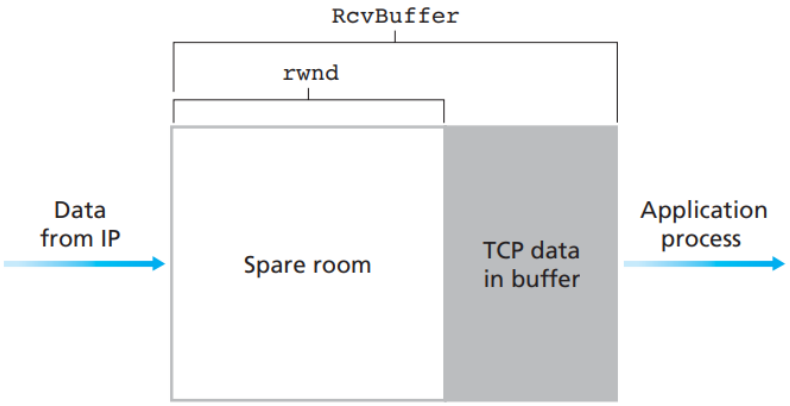


Figure 3.38 ♦ The receive window (`rwnd`) and the receive buffer (`RcvBuffer`)

TCP CONNECTION

Before data transmission, TCP establishes a connection between the client and server using a **three-way handshake**. It uses **SYN** and **ACK** messages to synchronize sequence numbers and confirm that both sides are ready.

Step 1: SYN

- The client sends a **SYN segment** to the server.
- The **SYN bit = 1**.
- The client chooses a random initial sequence number called **client_isn**.
- The segment contains no application data.

Step 2: SYNACK

- The server receives the SYN and allocates the required **buffers and variables** for the connection.
- The server sends a **SYNACK segment** back to the client.
- **SYN bit = 1**.
- **Acknowledgment number = client_isn + 1**.
- The server chooses its own initial sequence number, called **server_isn**.
- This means: *"I received your SYN and agree to establish the connection. Here is my sequence number."*

Step 3: ACK

- The client receives the SYNACK and allocates its connection resources.
- The client sends an **ACK segment** to the server.
- **Acknowledgment number = server_isn + 1**.
- **SYN bit = 0**.

- The TCP connection is now **established**.
- This third segment can also carry **client-to-server application data**.

In simple form:
Client → Server: SYN, Seq = client_isn
Server → Client: SYNACK, Seq = server_isn, ACK = client_isn + 1
Client → Server: ACK, ACK = server_isn + 1

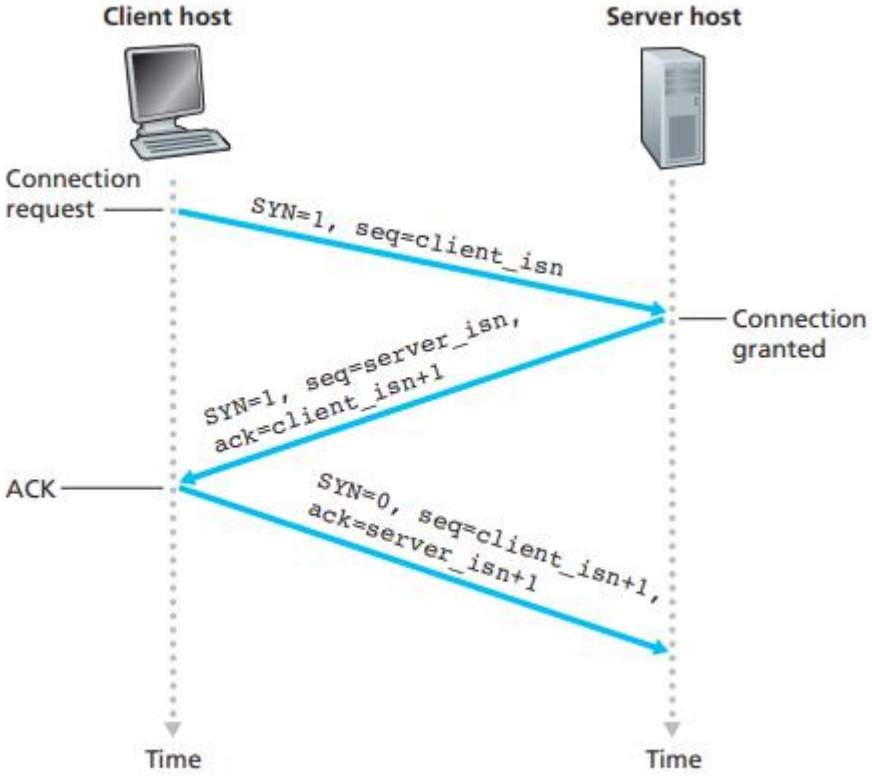


Figure 3.39 ♦ TCP three-way handshake: segment exchange

TCP CONNECTION TERMINATION AND STATES

Either the **client or server** can close a TCP connection. When the connection is closed, the TCP **buffers, variables, port numbers, and other resources** are released.

TCP Connection Termination

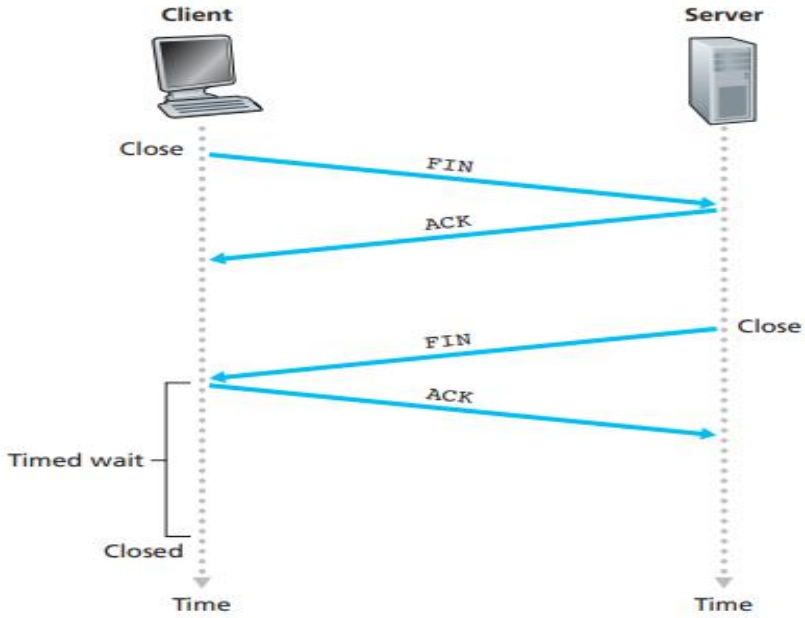


Figure 3.40 ♦ Closing a TCP connection

Suppose the **client starts closing** the connection:

1. **Client sends FIN:** The client sends a TCP segment with **FIN = 1**, indicating that it has finished sending data.
2. **Server sends ACK:** The server receives the FIN and sends an **ACK** to the client. The server may still send any remaining data.
3. **Server sends FIN:** After finishing its data, the server sends its own segment with **FIN = 1**.
4. **Client sends ACK:** The client receives the server's FIN and sends the final **ACK**.

After this, the connection is closed after the required waiting period.

TCP States – Client Side

The client normally moves through the following states:

CLOSED → SYN_SENT → ESTABLISHED → FIN_WAIT_1 → FIN_WAIT_2 → TIME_WAIT → CLOSED

- **CLOSED:** No TCP connection exists.
- **SYN_SENT:** The client has sent a SYN and is waiting for the server's SYNACK.
- **ESTABLISHED:** The connection is established. Both sides can send and receive data.
- **FIN_WAIT_1:** The client has sent FIN and is waiting for an ACK from the server.
- **FIN_WAIT_2:** The client has received the ACK and is waiting for the server's FIN.
- **TIME_WAIT:** The client has received the server's FIN, sent the final ACK, and waits for some time in case the ACK is lost and needs to be sent again.
- **CLOSED:** After the waiting period, the connection is completely closed and resources are released.

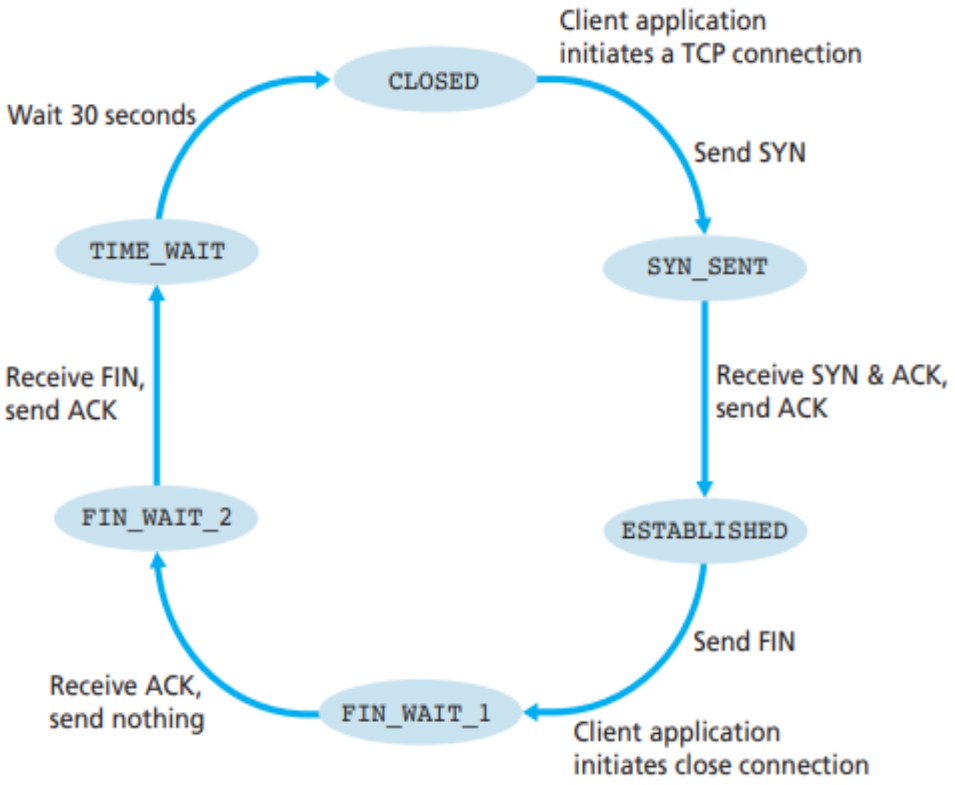


Figure 3.41 ♦ A typical sequence of TCP states visited by a client TCP

TCP States – Server Side
When the client starts the connection and later closes it, the server normally follows:
CLOSED → LISTEN → SYN_RCVD → ESTABLISHED → CLOSE_WAIT → LAST_ACK → CLOSED

- **CLOSED:** No connection exists.
- **LISTEN:** The server is waiting for a client's SYN request.
- **SYN_RCVD:** The server has received the SYN and sent a SYNACK.
- **ESTABLISHED:** The connection is established and data can be exchanged.
- **CLOSE_WAIT:** The server has received the client's FIN and sent an ACK. It can finish sending any remaining data.
- **LAST_ACK:** The server sends its own FIN and waits for the client's final ACK.
- **CLOSED:** The server receives the final ACK and closes the connection.

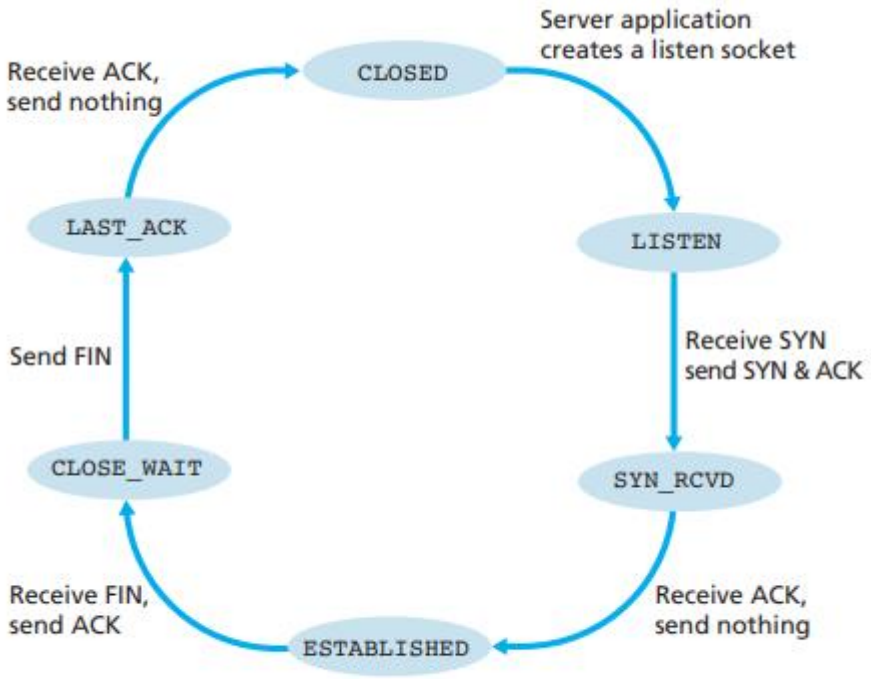


Figure 3.42 ♦ A typical sequence of TCP states visited by a server-side TCP

SYN Flood Attack
During the normal three-way handshake, the server allocates **buffers and connection variables** after receiving a SYN. It then waits for the client's final ACK.
In a **SYN flood attack**, an attacker sends a large number of SYN requests but does not complete the handshake. The server keeps resources for these **half-open connections**, which can exhaust its resources and prevent legitimate clients from connecting.

SYN Cookies
SYN cookies are a defense against SYN flooding.

- Instead of immediately allocating resources for every SYN, the server creates a special **cookie** as the sequence number and sends it in the SYNACK.
- The server does not keep connection information for that SYN.
- If the client sends a valid ACK, the server verifies the cookie and then creates the connection.
- If no ACK arrives, the server has not wasted resources on that request.

TCP Reset (RST) Segment

If a TCP segment arrives for a **port where no application/socket is available**, the host sends an **RST (Reset)** segment.

RST tells the sender that **there is no valid connection or listening application for that request**.

If a TCP segment arrives at a host but its **destination port or source IP address does not match any ongoing socket**, the host sends a **reset (RST) segment**.

For example, if a SYN is sent to **port 80** but no application is listening on port 80, the host sends an RST with the **RST flag set to 1**.

Meaning: "I do not have a socket for this segment. Do not resend it."

Nmap Port Scanning

Nmap can be used to check whether a TCP port is open on a target host. It sends a **TCP SYN** to the specified port.

Three possible results are:

1. **SYNACK received → Port is open**
   An application is listening on that port.
2. **RST received → Port is closed**
   The host was reached, but no application is using that port. It also indicates that the SYN was not blocked before reaching the host.
3. **No response → Possibly blocked by a firewall**
   The SYN may have been blocked by a firewall.

Nmap can also be used to examine **UDP ports, firewalls, application versions, and operating systems**.

TCP CONGESTION CONTROL

TCP provides **reliable transport** between processes. It also uses **congestion control** to prevent too much traffic from entering the network.

TCP uses **end-to-end congestion control** because the IP layer does not directly inform the sender about network congestion.

1. Congestion Window (cwnd)

TCP uses a variable called the **congestion window (cwnd)** to control the amount of data sent into the network.

The amount of unacknowledged data is limited by:

LastByteSent − LastByteAcked ≤ min(cwnd, rwnd)

- **cwnd** – congestion window
- **rwnd** – receive window used for flow control

For understanding congestion control, we can assume **rwnd is large**, so **cwnd mainly controls the sending rate**.

Approximately,

Sending rate ≈ cwnd / RTT bytes/sec

Therefore, increasing **cwnd** increases the sending rate, while decreasing **cwnd** reduces the sending rate.

2. How TCP Detects Congestion
TCP considers a **loss event** as either:

- A **timeout**, or
- Receiving **three duplicate ACKs**

When congestion becomes high, router buffers may overflow and packets may be dropped. This packet loss is therefore treated by TCP as an indication of congestion.

3. ACKs as a Signal

When ACKs arrive successfully, TCP assumes that the network is delivering data properly and **increases cwnd**.

TCP uses ACKs to control the increase of cwnd, so it is called **self-clocking**.

4. Basic Principles of TCP Congestion Control

TCP follows three main principles:

- **Packet loss → decrease sending rate:** A loss indicates possible congestion, so TCP reduces **cwnd**.
- **ACK received → increase sending rate:** Successful ACKs indicate that the network can handle more traffic, so TCP increases **cwnd**.
- **Bandwidth probing:** TCP gradually increases its sending rate until a loss occurs, then reduces the rate and starts increasing again. This allows TCP to find an appropriate available bandwidth.

5. Main Components of TCP Congestion Control

The TCP congestion-control algorithm has three major components:

1. **Slow Start** – rapidly increases **cwnd** when ACKs are received.
2. **Congestion Avoidance** – increases **cwnd** more slowly to avoid congestion.
3. **Fast Recovery** – reduces the impact of packet loss and helps TCP recover quickly.

In short:
ACK → increase cwnd → higher sending rate
Loss → decrease cwnd → lower sending rate

Slow Start

When a TCP connection starts, **cwnd (congestion window)** is initially small, usually **1 MSS**.

- **MSS (Maximum Segment Size):** Maximum amount of data TCP can carry in one segment.
- For every newly received ACK, **cwnd increases by 1 MSS**.
- Therefore, **cwnd approximately doubles every RTT**.
- Example: **1 MSS → 2 MSS → 4 MSS → 8 MSS**

When Slow Start Ends

1. **cwnd reaches ssthresh:**
   ssthresh (Slow Start Threshold) is the point where Slow Start ends and TCP enters **Congestion Avoidance**.
2. **Timeout occurs:**
   A timeout indicates packet loss. TCP sets **cwnd = 1 MSS** and starts Slow Start again. **ssthresh = half of the old cwnd.**
3. **Three duplicate ACKs:**
   TCP assumes a segment is lost, performs **Fast Retransmit**, and enters **Fast Recovery**.

Key point: Slow Start increases **cwnd rapidly** until **ssthresh** is reached or a loss event occurs.

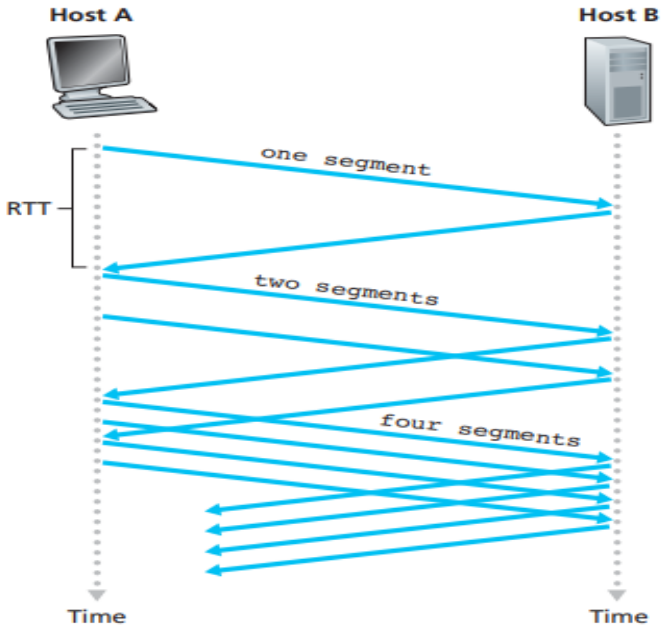


Figure 3.51 ♦ TCP slow start

CONGESTION AVOIDANCE

When **cwnd reaches ssthresh**, TCP enters Congestion Avoidance. Since congestion may be near, TCP increases cwnd **carefully** instead of doubling it.

- **Slow Start:** cwnd approximately doubles every RTT.
- **Congestion Avoidance:** cwnd increases by about **1 MSS per RTT** (linear increase).

Example: If cwnd = 10 MSS, it increases as **10 → 11 → 12 → 13 MSS**.
When Loss Occurs

1. Timeout

- TCP assumes serious congestion.
- **cwnd = 1 MSS**
- **ssthresh = half of cwnd** when the loss occurred.
- TCP starts **Slow Start** again.

2. Three Duplicate ACKs

- TCP assumes one segment is lost, but the network is still delivering other segments.
- TCP reduces **cwnd to about half** instead of resetting it to 1 MSS.
- It performs **Fast Retransmit** and enters **Fast Recovery**.

Key idea:
Slow Start → exponential increase
Congestion Avoidance → linear increase
Timeout → severe reduction
3 duplicate ACKs → less severe reduction

Fast Recovery

- In **Fast Recovery**, TCP increases **cwnd by 1 MSS** for each additional duplicate ACK received.
- When an **ACK for the missing segment** arrives, TCP:
  - reduces (**deflates**) cwnd to **ssthresh**
  - enters **Congestion Avoidance**.
- If a **timeout** occurs:
  - cwnd = 1 MSS
  - ssthresh = half of cwnd when loss occurred
  - TCP enters **Slow Start**.

TCP Tahoe vs TCP Reno

- **TCP Tahoe:** After **timeout or 3 duplicate ACKs**, sets cwnd = 1 MSS and enters Slow Start.
- **TCP Reno:** After **3 duplicate ACKs**, uses **Fast Recovery** and reduces cwnd roughly by half, then enters Congestion Avoidance.
- **Fast Recovery is recommended but not mandatory.**

Example: If cwnd = 12 MSS when 3 duplicate ACKs occur:

- ssthresh = 6 MSS
- **Reno:** cwnd becomes about 6 MSS and grows linearly.
- **Tahoe:** cwnd becomes 1 MSS and grows exponentially until ssthresh.

Key idea: TCP error control/retransmission and congestion control are different, but they are closely connected because packet loss triggers congestion-control actions.

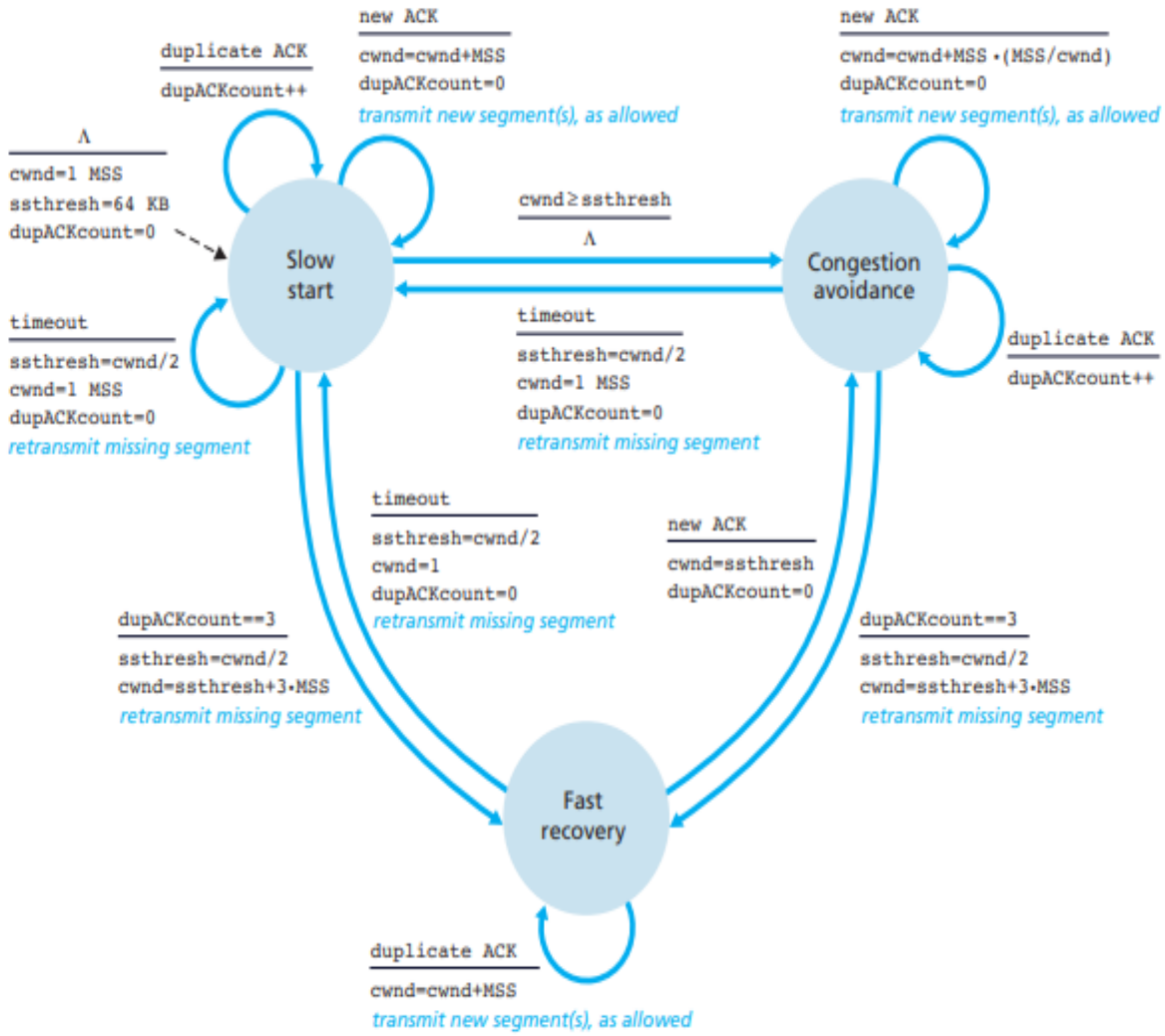


Figure 3.52 ♦ FSM description of TCP congestion control

TCP Congestion Control – Retrospective

After Slow Start, Congestion Avoidance, and Fast Recovery, TCP congestion control can be summarized as **AIMD (Additive Increase, Multiplicative Decrease)**.

AIMD

- **Additive Increase:** TCP increases cwnd by **1 MSS per RTT**.
- **Multiplicative Decrease:** When **3 duplicate ACKs** indicate loss, TCP reduces cwnd by **half**.
- TCP then starts increasing cwnd linearly again.

This produces a **saw-tooth pattern** in the congestion window.

Example:

8 MSS → 9 → 10 → 11 → 12 MSS
Loss occurs → 6 MSS
Then → 7 → 8 → 9 → ...

This is TCP's way of **probing for available bandwidth**: it increases the sending rate until congestion is detected, then reduces the rate and tries again.

Other TCP Algorithms

- **TCP Reno:** A commonly used TCP congestion-control algorithm.
- **TCP Vegas:** Tries to detect congestion **before packet loss occurs**, mainly by observing **RTT (Round-Trip Time)**.
- **TCP CUBIC:** Designed for **high-bandwidth networks** and is widely used in modern systems.

Key point:
AIMD = Increase cwnd gradually → detect congestion → reduce cwnd → increase again.

TCP Fairness
TCP aims to share the bottleneck link bandwidth fairly among competing connections.If **K TCP connections** share a link of rate **R**, each connection should get approximately **R/K**.

Why is TCP AIMD Fair?
- In Congestion Avoidance, cwnd increases by **1 MSS per RTT**.
- When loss occurs, cwnd is reduced by **half**.
- Repeated AIMD operation makes competing connections move toward an **approximately equal bandwidth share**.

Fairness and UDP

- TCP reduces its rate when congestion occurs.
- **UDP has no built-in congestion control** and may continue sending at a high rate.
- Therefore, UDP traffic can **crowd out TCP traffic**.

Fairness and Parallel TCP Connections
- An application can use **multiple TCP connections** simultaneously.
- More connections can give that application a **larger share of the bottleneck bandwidth**.
- Example: If 10 applications use one connection each, each gets about **R/10**. An application using 11 parallel connections can get a much larger share.

Key Point
TCP AIMD tends toward fairness, but **UDP traffic, different RTTs, and parallel TCP connections can cause unequal bandwidth sharing**.

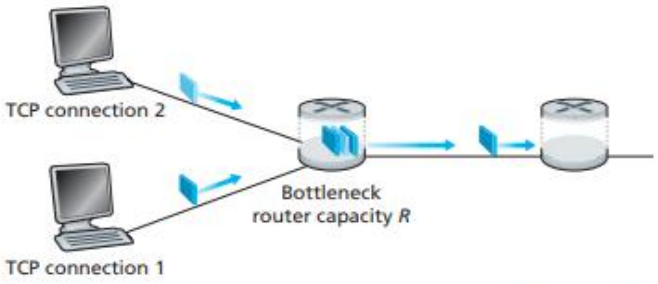


Figure 3.55 ♦ Two TCP connections sharing a single bottleneck link